高德本地生活分析 Agent 的 AI Native 实践全景（万字长文回顾）

原创 钟雨洁 阿里技术 2026年9月30日 03:08 浙江 16人



这是2026年的第 66 篇文章

（ 本文阅读时间：约 30 分钟 ）

FY27 的 S1 过去了。回头看这半年，像一场拉长的复盘：一个问题刚以为讲清楚了，下一个问题又摆到桌上。这个 S1，作者主要在建设**高德本地生活分析 Agent**。原以为分析流程跑通，就能往前迈一大步，后来才发现，**口径、知识、推理和交付**，每一处都有需要重新想清楚的问题。这半年确实不轻松，但也正是在这些反复里，慢慢看清了两件事：系统应该怎么建；作为 BI，应该把精力放在哪里。

> 💡 全文约 1.2 万字，预计阅读 30 分钟，可按需选读。
>
> 想了解背景与架构，看第 1—3 节；想看具体实现和踩坑经验，直接进入第 4 节；关心 AI 时代 BI 的定位与个人成长，可以从第 5—6 节读起。

这篇文章整理了这段实践的全景：从**整体架构，到分析流程、知识与语义治理、经营分析智能体**的具体建设，也记录了踩过的坑、做过的取舍，以及这些经历怎样改变了作者对 BI 的理解。到这个 S1 末，这套体系支撑了本地生活多个业务单元的例行经营诊断，全行业经营简报一小时内产出。希望其中一些经验，能帮正在做类似探索的同行少绕一点路。

01
背景与项目缘起：
报告交付之后，经营问题还在继续

作为 BI，会很熟悉一种场景：收入下降了，查数、核对、拆解，终于把分析交了出去。业务接着会问，这个变化要不要处理，先动哪个环节，手头正在做的事情有没有用？报告写完了，讨论才刚刚展开。

这些问题也是我想参与的。但日常工作里，大量时间花在澄清需求、确认口径、取数和核对上。业务、BI 和数据开发一起完成一份分析；换个问题，又要重新找人、找材料。许多经验明明积累下来了，下一次还是得再讲一遍。

自 S1 起，高德推进 BI AI 化，方向正好对着这些问题：**让业务能更自主地获取可信数据，让分析更深入地参与决策**。我负责的，是其中**面向本地生活业务的数据分析智能体**。

在这个项目里，我的做法是把已经弄清楚的业务关系和分析方法留下来，下一次直接用：**业务知识整理进知识库，分析方法写成 Skills，让 Agent 调用数据和工具完成分析**。省下的时间，可以回到那些报告之后还没谈完但却最需要人参与的问题上。

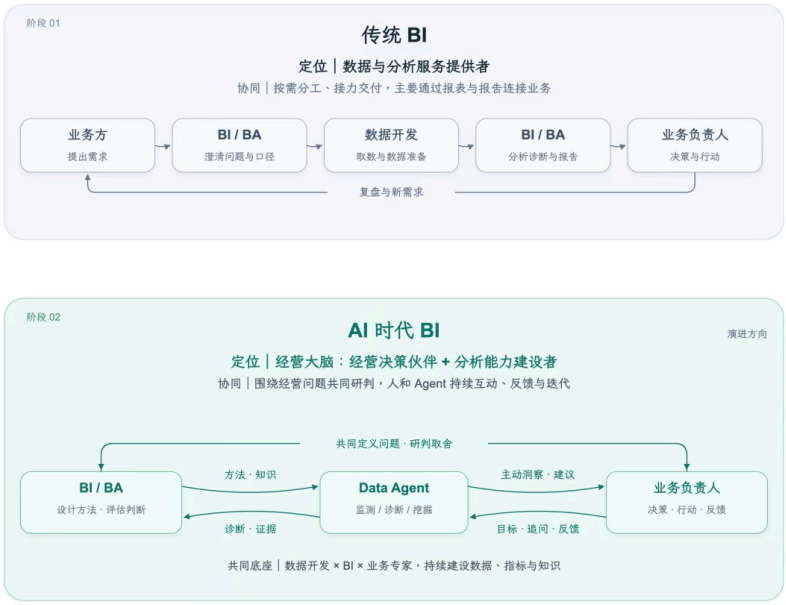


图1 AI时代的BI工作与协同边界 vs 传统时代

下面想分享的，就是这段建设经历：哪些地方跑通了，哪些地方返过工，以及这些返工怎样改变了我对 BI 的理解。

02
L0–L4 能力分级：从回答一个数，到支撑一个判断

要交代这段建设，得先说明白我们手里这件事做到了哪一步。团队内部讨论“为什么下降”这类问题时，需要一把对齐能力的尺子：结合行业调研和业务场景，**我们形成了 BI 能力的 L0–L4 分级**，用它来对齐进展——现在能做到哪一步，下一步还缺什么。

团队早期主要处于 **L2 智能问数阶段**。用户问“上周收入是多少”，系统准确理解口径、找到数据并返回结果，就已经能解决30%以上的需求了。

再追问一句“为什么下降”，需要的能力就不同了。



图2 智能分析演进路线

按这套分级看，我认为我们现在**绝大部分任务已具备 L3 的特征**，开始回答“收入为什么下降”。系统要沿业务链路排查，并拿出支持解释的证据。

组织内**部分场景已经走到了 L4**，把分析接到决策与行动上。这里的 **L4 不等于无人化：系统自主完成分析并给出建议，人参与关键取舍，并带回执行反馈**。本地生活分析 Agent 的实践主要在做 L3 的分析建设，也开始向经营研判延伸。至于自主执行、业务反馈怎样回到下一次分析，还在后面的建设计划里。

而从 L2 跨向 L3，首先要填的就是这个坑：**让系统真正理解一门具体生意里的“下降”在说什么**。这也是本地生活数据分析 Agent 的第一步。

03
三层能力框架总览：把经营问题带进建设选择里

我最初从广告经营分析切入，后来扩展到休娱丽人、生活服务等到店行业。第二个行业的接入比预想中更费力：例如同样是“收入下降”，收入逻辑、指标口径和分析路径都可能不同，已有经验不能直接搬过去。

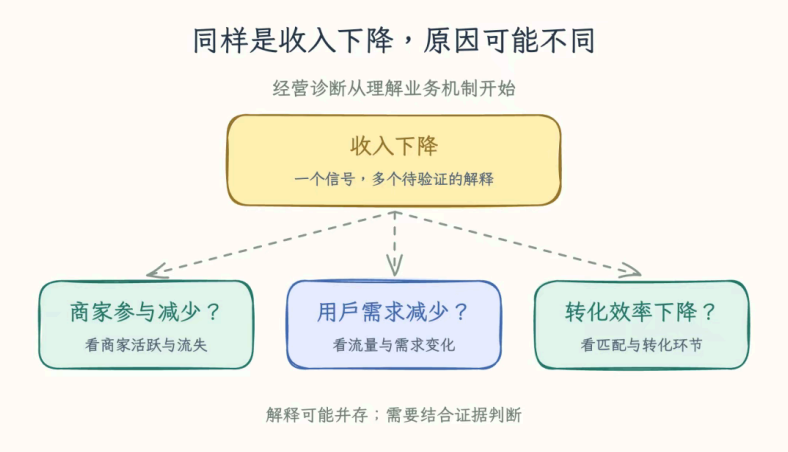


图3 同一个变化，可能对应不同的经营问题

每接一个行业都重新做一遍，也不现实。我的主要设计和建设，**把共用方法与行业差异整理进三层框架**。



图4 整体建设框架：当前重点是知识、分析与主动触达；追问权限、跟踪复盘和反馈闭环仍在完善

3.1 基础底座：让经验能被下一次分析用上

先从最底下这层说起，从基础底座看分析依据。分析师看到指标，已经知道它在说谁、按什么口径计算、和哪些动作有关，数字本身并不包含这些理解。我需要把它们整理成共享知识，连上指标与数据来源，供 Agent 每次分析读取。

知识还会变。一次分析用了哪版口径、哪份材料，出了分歧得能找回来，所以我让任务使用**固定版本的知识快照**。后续修订知识、管理权限，也需要能追溯影响。

3.2 分析能力：顺着经营问题组织方法

底座解决“分析依据什么”，中间这层要解决“怎么分析”。比如，定位到某个城市贡献了较大降幅，还不能解释原因。系统得知道接着查什么，怎样结合已有动作研判。我用 Skills 写分析方法，用 Pipeline 连接步骤，先做经营状态识别、归因诊断和策略研判，再向跟踪复盘延伸。

这里一直有个取舍：哪些步骤写进工程化程序，哪些交给模型判断？**我的做法是让程序负责计算和明确规则，模型结合知识解释业务，人参与关键取舍。**具体的边界，是在返工中逐渐划清的。

3.3 产品交互：先把重要发现送到人面前

分析做出来，还要考虑业务会怎样用。我选择**先做主动触达**，因为业务负责方需要及时看到重要变化，省去查找和筛选。因此，**推送摘要先给判断，完整报告再展开解释（展示什么、以什么样的形式展示，真的很重要）**。

如果面向一线业务人员和分析师，我会作不同的选择。他们常常带着具体问题来，查完还要补条件、比数据、验证想法，**交互问答与自助分析**则会更适合这类工作角色。

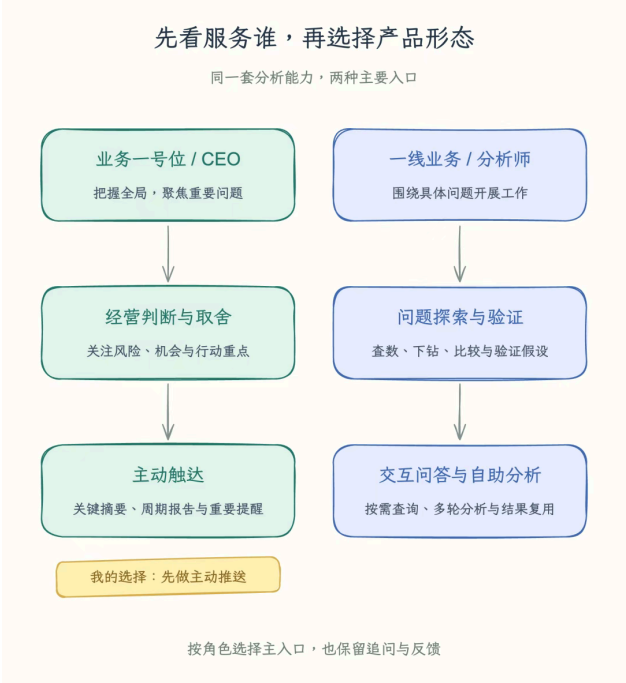


图5 按主要任务选择入口，主动触达与按需探索可以衔接

当然，业务负责方收到推送也会追问。但问答能力目前卡在**权限衔接**：用户从一个行业追问到另一个行业，或从全国总盘追问到某个城市时，系统都要确认他能看哪些知识、数据和结果。我们打算先支持已授权报告内的追问，再扩展到新的取数和分析。

框架搭好，只是建设的开始。真正动手之后，最费力的部分常出在那些看起来已经做完的环节里——下一节讲的三段返工，都从这个框架里长出来。

04
三段实践，三次重新理解

S1 做了很多尝试，我想挑三段相互交叉且最重要的经历来讲：**分析执行、知识复用、经营判断**。

每一段都从一次返工开始，也以一次对框架的重新理解收尾。

实践	最初遇到的问题	调整后的建设重点
分析执行	归因算出来了，报告仍然浅	保留证据、明确分析路径、用案例回归
知识复用	文档有了，换个任务还是用不好	按需加载、独立修订、追溯指标口径
经营判断	数字正确，业务判断仍可能出错	补齐背景、释放推理空间、审阅语义

三段实践在框架里的位置并不相同：**分析执行主要落在分析能力层，知识复用落在基础底座，经营判断则由二者共同支撑**。

4.1 分析流程与工程化：让分析跑起来

最先要解决的，是让经营状态识别和深入归因诊断跑起来。分析师平时顺手完成的检查，交给 Agent 后都得重新说清楚。第一个很具体的麻烦，就出在一个简单的除法上。

4.1.1 指标校验与分析规则设计： 同一指标差了 12% ，先回到取数环节

一次行业接入中，用总量除以对象数量，得到的平均值与直接查询结果相差约 10%。追查发现，分子与分母来自两套数据源，统计范围没有对齐。我们统一来源与口径，核验“总量＝对象数量×平均值”，闭合后才继续归因。

数字对齐后，归因才能继续。但算出平均值是主要下降因子，还得解释它为什么下降。我也开始检查，扫描规则有没有把不同的业务情况混为一谈：

- **目标与趋势。** 达标仍要检查趋势和内部因子；未达标但趋势稳定，不能直接判为经营恶化。
- **变化与节奏。**跨月周报除比较上周，还看上月同一周和去年同期，避免把月末到月初的正常变化当成异常。

这些区别，分析师通常会结合背景判断。交给系统后，计算口径和判断条件都得明确，否则后面的解释很容易跑偏。

4.1.2 上下文设计与报告质量校验：归因已经算出来，为什么报告还是浅？

后来遇到的一个问题更隐蔽：Agent已经算出了单维度和组合维度归因，报告却没有呈现。沿着“引擎输出→报告组装→写作输入”查下去才发现，一处只留前三项，另一处把组合分析移进了备查区。写作模型只读正文，它根本没见到这些证据。

找到原因后，我在输入里保留重要组合的贡献度、异常判断和定位线索，再用关键证据清单核对成稿，遗漏就退回修改。至于模型怎样综合这些组合、怎样组织表达，我留了空间，没有要求每条材料都套同一句式。

沿着写作输入，还查出了“报告像分析日志”的原因。程序会先生成带内部标签的草稿，模型润色时也把“机制已坐实”这样的措辞带进了报告。后来我让模型**直接读取结构化事实与归因结果完成诊断写作，再用业务表达样例（Golden Cases）校准措辞**，检查标签残留。

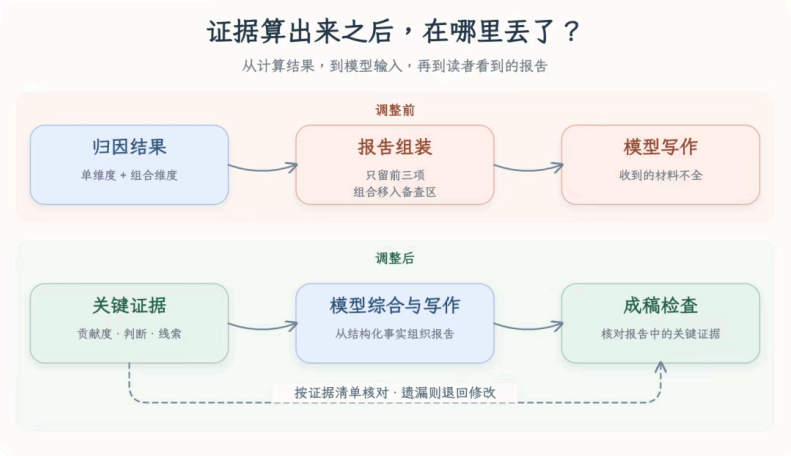


图6 计算完成、进入上下文、出现在成稿中，是三个需要分别检查的位置

4.1.3 Agent Handoff 设计：两个开发 Agent 接力，交接文件也会成为负担

前面这些修改，开发过程本身也在用 Coding Agent。考虑数据敏感性，行业接入的一段开发中，我让两个 Agent 交替承担方案审查、实现和验证，通过共享文件交接。“场域枚举用英文，不能直接传中文”这样的接口细节也会记下来，免得换个会话又踩一次坑。

问题是，交接文件越写越长，当前状态和历史过程混在一起。接手的 Agent 要先从中找出哪些决定还有效。我把入口收拢成几十行当前摘要，留下**有效决策、待办和验证状态**，历史过程另外归档，需要时再读。

交接之后还要检查改动有没有影响原有能力。接入新行业时，Agent 会重跑已有广告案例，我再审阅分析和业务表达。审阅意见不只是验证，也会暴露新的缺口——下一小节的能力补齐，就从其中一条开始。

4.1.4 反馈驱动的能力迭代：一句“没有继续下钻”，要改到哪一层？

审阅中就有这样一条反馈：报告定位到某个转化环节便停住了，没有继续下钻。这次材料没有丢，缺的是后续分析路径。我把分析继续拆到对象状态和使用过程：从纳入统计的对象追到实际使用对象，再分别检查首次使用与持续使用，区分“尚未开始”和“开始后未能持续”。

前一次“归因算出来却没写出来”，要修**证据传递**；这次则要补**分析路径**。再收到“分析不够好”的反馈，我会**先看问题发生在哪一层，再决定补方法、补知识还是改实现，最后拿案例重跑**。

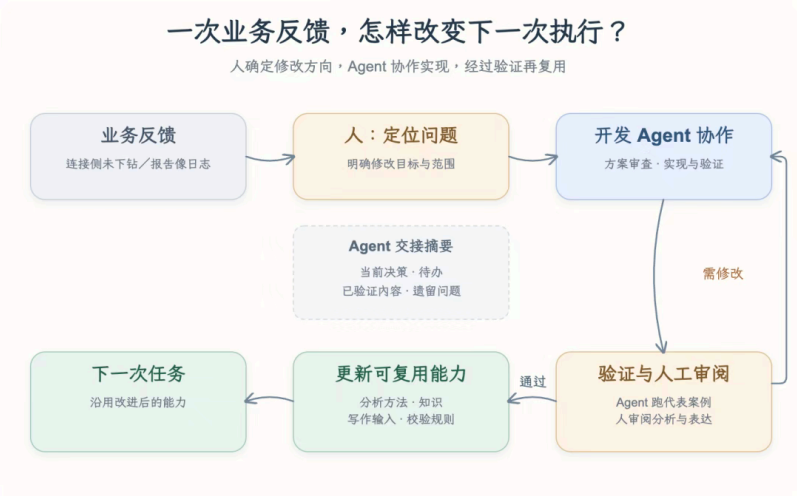


图7 开发过程中的反馈循环；业务决策与执行的闭环，还需要后续建设

几次返工下来，我会严格检查证据有没有传到、该查的路径有没有走到；材料齐了以后，怎样综合和表达，就让模型多作判断。

> 💡 回头看，这一段让我重新理解了“跑通”的含义：跑通不是流程走完，是证据传递和分析路径由工程兜底，深度推理、综合与表达留给模型判断。工程把确定性做满，模型把判断力放开。
>
> *（为保证数据准确性，在垂直分析Agent中做了特定分析路径的工程兜底）*

4.1.5 Demo：一份经营诊断月报，怎样组织分析

一路补到这里，一份诊断报告怎么组织，才算有了样子。读者最终看到的报告，大致是下面的样子。这份改编示例呈现了增长质量判断、原因比较和策略取舍。



图8 能力展示示例，内容经改编（业务信息与数值以脱敏占位展示）

要让这样的分析换个行业也能做，还得补上业务前提。哪些对象被纳入统计，怎样算开始使用，怎样算持续使用——这些分析师默认知道的区别，Agent 得从哪里获得？

4.2 知识库与语义治理：让经验跨业务复用

为了建语义库，我回头去整理线上分析师知识库、业务规划和复盘文档。材料里有收入机制、指标口径，也有分析师的经验，但它们原本是写给人看的。**要让 Agent 用得上，先得处理知识怎么提炼（蒸馏），再考虑放在哪里（结构）、怎样持续更新（保鲜）**。

4.2.1 知识蒸馏与按需加载：文档收齐了，为什么分析还是用不好？

拿一份复盘来说，里面可能同时写着经营现象、原因判断、当期策略和历史口径。压缩成摘要，Agent 能知道这次发生了什么；下一次遇到类似问题，具体先查什么，摘要未必说得清。

所以我先把**事实、机制、策略与口径**拆开，再按分析时的用途整理成三个层次，输出规范另外维护：

- **行业知识：读懂这门生意。**行业文件保留业务模式、核心公式、口径和诊断起点；季度策略、事件和口径变更另放主题文件，区分稳定规律与当期情况。

- **分析框架：知道沿什么顺序排查。**根据业务经营框架组织问题，形成分析路径，收入、交易规模等结果性指标按照业务指标树拆解到影响因子。
- **分析方法：把判断落实为步骤。**将 **20 多种方法按任务归并为 9 类**——定义、诊断、校验、复盘等——细粒度方法放到适用场景下，写清前置信息、执行步骤和常见陷阱。

这里面最需要花心思的，是**把经验里省略的步骤补出来并让 Agent 持续理解和学习**。比如某项使用率下降，“按对象分层”还不够具体：先看纳入统计的对象总量是否扩张，再看实际使用对象的绝对量；比较各群体的占比和使用率，区分构成变化与群体自身变化，必要时再追查新增对象为什么没有开始使用。这样写，Agent 才知道下一步查什么。

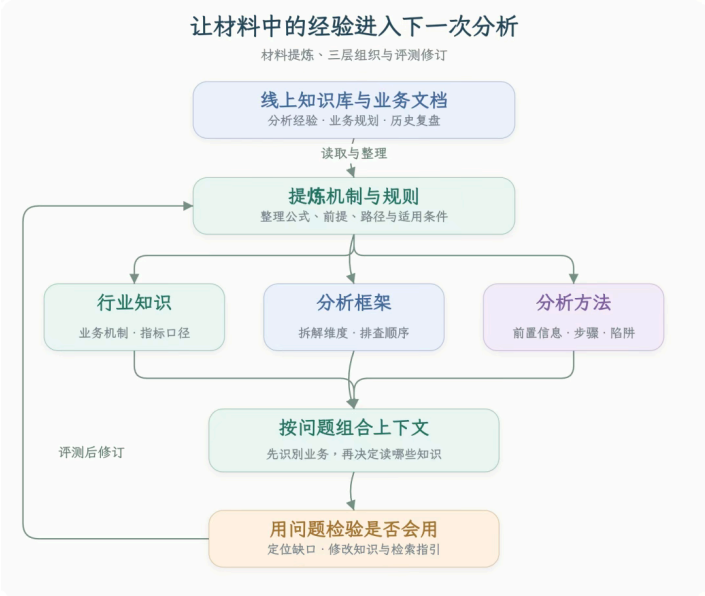


图9 按问题组合知识，无需每次读完所有材料

把内容写清楚后，怎么存放又成了问题。最初按知识、分析、表达分别存，一次任务要跨十几个文件；改成按行业收拢，文件又随着事件追加越写越长。最后我**拆成“稳定行业文件＋时效主题文件”**，一次重组将行业文件缩减大半，变化内容转入主题文件。问口径读行业概要，追问原因时再追加框架和方法。

文件搬完，还不能算完成。评测时发现，入口仍指引 Agent 去旧章节找口径。我又**同步修订检索指引，检查旧值残留，用同一批问题复测。内容、入口和评测问题都得跟着改，Agent 才能读到正确材料**。

4.2.2 共享知识建模：每个 Skill 都存一份，知识怎么保持一致？

整理单个应用的知识时，还能靠逐项检查解决问题。但上一节做的只是单应用内部的组织，指标定义仍散在各 Skill 的实现里；应用多了，同一指标在不同 Skill 里各存一份，修订一处，其他地方未必跟着变。我于是**把知识从具体 Skill 中独立出来，用共享底座维护定义、关系和来源**。

前面**按模型阅读的需要分层，解决的是“分析时读什么”**；这里**按维护的需要分层，解决的是“改了影响什么”**。两套分层视角不同，指向的是同一批知识。按知识支撑的业务判断，我归纳

为三层：

- **业务语义：把对象和数字认对。**有商品门店、广告投放商家分别是谁，广告转化渗透率用什么分母，连接哪些公式和数据来源。
- **诊断知识：把变化转成可验证的问题。**经营机制、拆解规则和比较基准帮助提出假设，再用当次数据验证。
- **策略与经验：把判断放回业务背景。**目标、行动、执行记录与历史案例，说明已有动作回应了什么约束、经验在什么条件下适用。



图10 上方是知识的支撑关系，下方是构建过程；业务范围、来源与版本贯穿各层

要让这样的结构真正可用，还得**把几个细节落实到存储和读取里**：

1. **对象可独立修订。**指标、公式、业务对象各有稳定编号与别名，形成可查询的节点；每项属性保留来源位置。“项目名称”应归为维度，不能因为出现在表里就当成指标。
2. **关系有类型，也有身份。**指标连接公式和来源字段，经营关系连接业务对象。编译时检查端点、引用与来源，再固定版本。
3. **读取时恢复业务上下文。**先确定业务单元与任务，再沿相关关系取回定义、机制和证据。各分析应用按绑定版本读取，便于复现当次分析；共享底座与各入口的更新衔接，是下一步要补的。

关系的身份就曾经丢过一次：**上百条业务关系**和两端对象都进了图谱，用原材料的关系编号查询，却 **0 条命中**。编译保留了连接，没保留关系记录自身的身份。补上独立关系节点和原始编号后，这些业务关系才全部能唯一找回。只看“关系已经入库”，发现不了这个问题。

共享底座、业务诊断和经营判断逐步积累了这些知识，彼此有继承与导入关系，按各自范围统计。



图11 三处知识按各自范围展示，内容可以继承与复用

4.2.3 指标血缘与语义更新：看板口径变了，Agent 怎样跟上？

还有一种变化发生在知识库外面：看板的取数逻辑改了，Agent 读的口径却可能没变。看板中文名、数据集字段别名和 SQL 表达式又不总是一一对应，单看名称找不出差别。为此我构建了**指标口径更新 Pipeline，以 FBI 业务顶层看板口径为权威源，从看板和 SQL 反向追踪**：

- **追到计算来源。**从组件找到数据集，解析 SQL 的语法结构，追踪临时查询结果、别名、聚合与表达式，直到来源表和字段，并回查物理表与字段真实存在；保留筛选条件与解析缺口。
- **按技术口径识别指标。**表达式、来源字段、聚合方式、时间粒度和筛选范围共同生成稳定标识。同名但口径不同的指标分别保留，等待核对。
- **生成可查询的知识包。**数据集、指标、来源表各自形成 Markdown 节点，由索引连接；保存公式、血缘、证据及确认状态，供分析任务重复读取。

例如，一单可能有多条明细时，两个都叫“订单量”的字段就有不同含义：

表达式	实际计数对象
`COUNT(order_id)`	非空记录；一单多行时会重复计数
`COUNT(DISTINCT order_id)`	去重后的订单编号

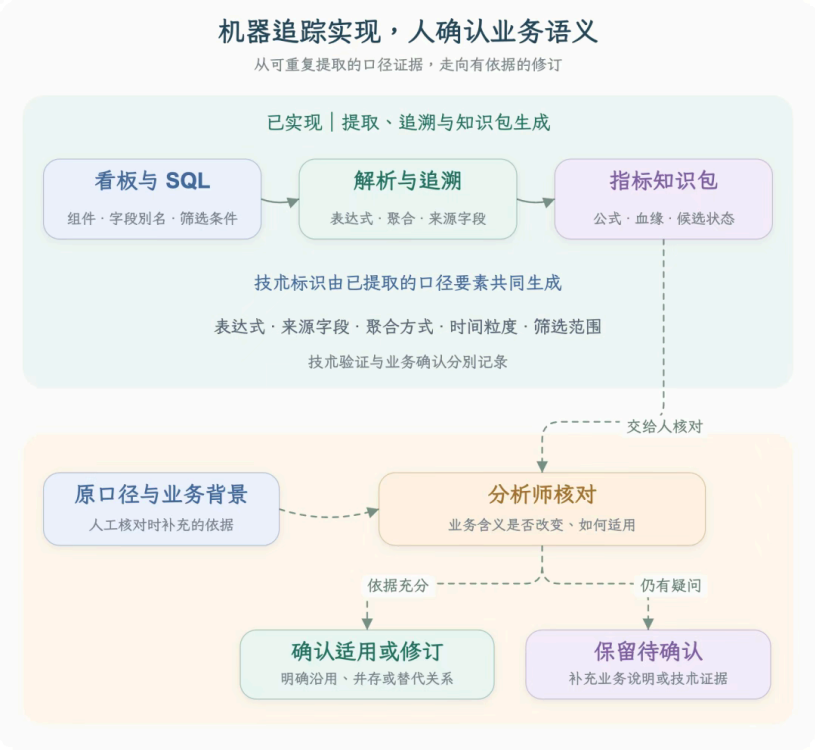


图12 实线为已实现的提取与知识包生成；人工确认及共享库同步仍需衔接

重新提取后，公式或筛选范围的变化就能被找出来。但表达式变了，业务含义是否也变了，新旧口径应并存还是替代，**还需要分析师判断**。目前候选与证据提取已经做出来，自动比对历史版本、确认后同步共享库，仍在建设中。

做过这部分以后，我会把入库的知识再放回能否找回、会不会用，单看入库结果并不知道。分析任务里检查。

> 💡 能否找回、会不会用，单看入库结果并不知道。
> 入库不算完成，被分析任务正确用上才算。到这一步，知识才从文档变成资产。

4.3 经营分析智能体：让分析支撑业务取舍

前两段实践解决了口径和知识的问题：数字能算对，经验能复用。但还有第三个问题，比前两个更靠近业务——数字正确的报告，交到业务一号位手里，能不能支撑他作判断？这段实践就从这里开始。

业务一号位读报告，会把分析放回正在做的事情里：增长靠什么维持，已有策略有没有回应真正的约束？如果系统只看指标变化，不知道业务正在改变哪个环节，就很难参与这样的讨论。我开始**把经营背景带进分析，也给模型更多综合判断的空间**。

4.3.1 经营上下文与输入建模：指标之外，业务正在做什么？

先补输入。早期上下文主要是业务映射、收入分类和指标清单，对团队正在做什么交代得不够。因此我补入经营事实，记录结果与诊断；**用经营卡片记录业务经营进展、目标偏差和阻塞；再用策略账本串联业务行动、投入与反馈**。稳定商业模式和年度规划从知识库读取。输入

之间也要划清边界：**经营简报只做关联业务策略动作的经营结果验证（Facts），策略简报只证明策略的身份、动作与执行（Facts），诊断结论以 Agent 定期体检扫描的数据结果为准（Insights 来源于 Agent 判断）**。

材料多了，能不能一起使用，也要先检查。业务对象、报告周期和来源版本需要对齐，还要能定位原文。尤其容易混淆的是时间：本月更新的记录可能引用旧周期，“计划扩大覆盖”也不能直接解释本月增长。



图13 文档更新时间、事实发生时间和行动状态分别处理

输入补齐后，我还尝试把单行业结果汇成更高层经营总览。这一版做出了工程预览，离正式交付还差一些，但它让我看清：**跨行业汇总需要一套自己的准入、判断和展示约束。**重新看产品目标，我重新调整了优先级，决定先收回默认汇总层级，把各业务一号位需要的报告做好。

4.3.2 Skill 驱动的经营推理：规则越来越多，分析为什么没有更好？

回到单份报告，也有需要做减法的地方。早期分析只接着上游异常往下走，为了保证报告呈现稳定性（毕竟是面向一号位的），我又给报告加上一批硬约束：固定三个议题、每条议题至少两个数字、卡片配额、句长限制、策略效果分级，等等。报告看起来更规整了，判断却没有跟着变好。

一次检查里，泛化的“指标反映上升变化，需结合相邻环节复核”能通过深度检查；“尚不能确认……导致……”明明是在保留意见，却被因果词黑名单拦住。门禁检查抓住了一个词，没有读懂整句话。

于是把这批硬约束（固定议题、卡片配额、句长限制、因果词黑名单等）干脆地全部做了减法，把精力转回 Skill 里的分析方法：

- **先看经营全貌。**联看收入、规模、订单和收益，增长质量同样值得分析，异常清单不再是唯一入口。
- **再沿适用机制展开。**由商业模式选择经营链路，结合团队、类目、城市等结构寻找解释。
- **让策略参与判断。**行动回应了哪个约束，执行与指标变化是否互相支持，有没有反向信号？有依据可以作定性研判，精确贡献则另需测算。



图14 三种场景（模式）仅作举例，复杂业务可以涉及多种模式

调整后的报告开始联读供给、交易承接、收益质量和策略，不再只写三个固定议题；另一个样本还区分了“单兵效率改善”与“总产能恢复”。代码把问题预先选得太满，模型就只剩下润色的空间了。

4.3.3 事实绑定、语义审阅与成稿分工：数字引用没错，为什么结论仍然错了？

不过，判断空间放开以后，错误也可能藏得更深。一次初稿把**全部纳入统计的对象当成尚未开始使用的对象**，推断转化不足。数字来源正确，对象范围却错了：总量中可能包含已经使用的对象，判断转化前，必须先明确待使用对象的范围与观察周期。另一个样本第一次修订后仍残留同类错误，又做了定点修正。

这样的错误，**光核对数字查不出来，继续加规则也防不胜防**——上一小节的返工已经证明了这一点。我换了一个思路：**经营判断的可靠性，不能指望单次生成一次做对，要靠一套与判断空间匹配的质量工程**。这套工程回答两个问题：怎样让每类错误都有对应的一层检查接住，以及出错之后，怎样让修正付得起代价。

先看防错。检查按错误类型分成三层，各管一段：

- **事实绑定，防止引用错。**数值、单位、期间和比较状态由上游提供，模型引用事实标识，程序还原数值；新增计算先回到计算环节，结果核验后再用于判断。
- **分析与业务审阅，检查理解。**模型形成结论、依据、适用条件与短稿，程序检查关键问题是否覆盖、引用能否追溯；我再结合业务知识审阅对象混用、错误机制和遗漏的反证。
- **成稿保真，防止交付变样。**第一轮工程检查通过后，成稿模型按内容块 ID 编排已确认的内容与图表；第二轮检查遗漏、新增与改写，确保判断和条件没有变样。

三层分工也厘清了成稿与诊断写作的区别：经营判断在分析阶段已经完成，后续成稿只组织已确认的内容，不再作新的解释。否则，一次看似顺畅的润色，也可能把判断成立的条件改掉。



图15 不含返修时，主路径为分析、成稿两次模型调用；业务能力另用同周期、同快照的配对评测检验

再看容错。批量交付里，返修是流程的常规一环：一次交付中，**15 个生成单元组织成了 11 份报告**，我逐篇审阅，让模型修订语义问题后再复核。既然返修一定会发生，设计要回答的问题就变成了：返修一次，代价多大，影响多远？一个运行样本有 **731 条来源、720 条事实、171 个指标族**，如果只错一个引用也要把全部材料重新送给模型，调用开销大，还可能把原本正确的段落一起改坏。为此做了两件事：

- **输入做减法，让修正按需取材。**首次输入先压平共享引用和重复包装，最大引用深度由 **13 层降到 4 层**，事实与引用仍可完整还原；返修时按错误字段、引用关系和历史比较选取上下文，保留相关知识、策略与反证，限定可改段落。一个样本的返修来源由 **731 条收敛到 179 条**。
- **按修改的影响半径决定重跑范围。**判断变了，重新检查分析和后续交付；只改版式，就保留原判断；遇到工程误报则修程序、复验留存输出，不让模型反复改正确内容。修改的影响有多大，重跑的范围就有多大。

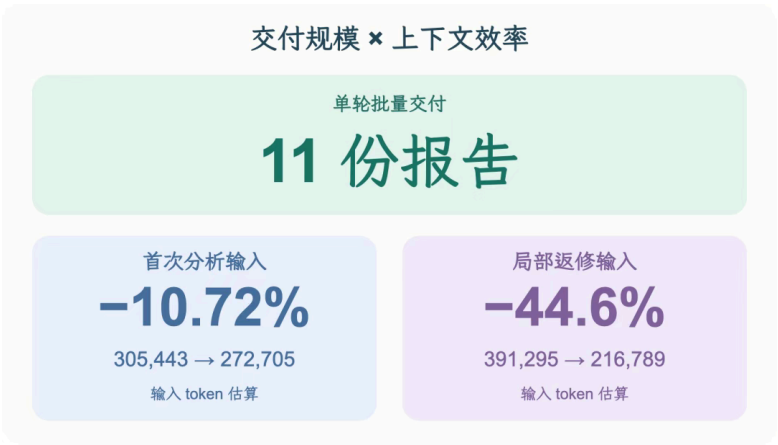


图16 两项对照中的输入 token 估算减少量

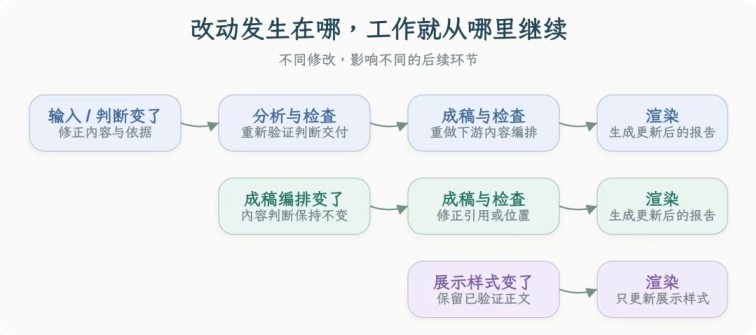


图17 按修改影响选择重跑环节

实际跑下来，这套机制还不算轻松：一份月报经过 **3 次分析、1 次成稿**完成，模型调用累计约 **11.1 分钟**；同一轮验证的另一业务样本三次尝试后，仍因表达错误停止。怎样在有限的重试里修对问题，还要继续打磨。所以设计时，我开始把“出错以后怎么办”当成一等公民：**保留哪些依据，改动影响哪里，哪些检查要重做**——这些细节决定了系统能不能持续使用。

> 💡 给模型判断空间的前提，是质量工程兜得住——每类错误有对应的一层检查，每次修正知道改到哪一层为止。判断空间决定报告的上限，这套工程决定报告的下限。

4.4 知识和能力沉淀：三段实践，留下了哪些成果？

三段返工讲完，也该交代一下它们最后攒下了什么。我建设的系统逐渐支撑了团队的批量交付，留下了这些成果：

- **覆盖范围：本地生活业务单元全部接入。**各业务使用适配的商业模式、口径与分析路径；前面的 11 份报告是某次交付批次，两者统计对象不同。
- **产出时效：全行业一轮报告 ≤1 小时。**过去每业务单元人工约 1 小时，一轮约 22 人时。以人工逐份完成、系统按 1 小时折算，批量产出效率约为原来的 **22 倍**。
- **知识资产：从文档积累到关系化维护。** 共享底座连接业务对象、经营机制与分析经验，诊断和经营判断按需使用，减少重复梳理。
- **指标语义：连接定义、计算依据与来源。**指标口径更新 Pipeline 提供变更证据，为持续维护这份资产打下基础。



图18 知识和语义规模

这些口径、方法和业务理解留下来以后，接入下一次分析就有了基础。已有能力还能用在哪些场景，是下一节要展开的问题。

05
AI时代重新理解 BI 的定位：从场景能力到经营大脑

前面三段实践，留下了覆盖、时效和一批能复用的知识与方法。但故事不能停在这里：**这些能力主要是围绕单个业务单元的经营体检建起来的，而经营问题常常不止于一份报告**。经营体检之外，**供需匹配、供给质量、客户变化、竞争动态和市场趋势追踪**，都值得继续做。最初看，它们是几个新的分析任务；放在一起看，又会发现问题经常相互关联：供给出了变化，可能和需求有关，也可能和客户预算、竞争动作有关。

如果每个 Agent 只完成自己的报告，这些联系仍然要靠人重新整理。

我开始想，**能不能让它们围绕同一个经营问题协作，把分散的线索接起来，看看有没有单份报告里没发现的规律**。对“经营大脑”的思考，就是从这里继续展开的。

我认为，**BI 应该持续理解业务正在发生什么，判断哪些问题值得优先关注，再组织需要的知识和分析能力**。这也是我们设计经营大脑时的出发点。

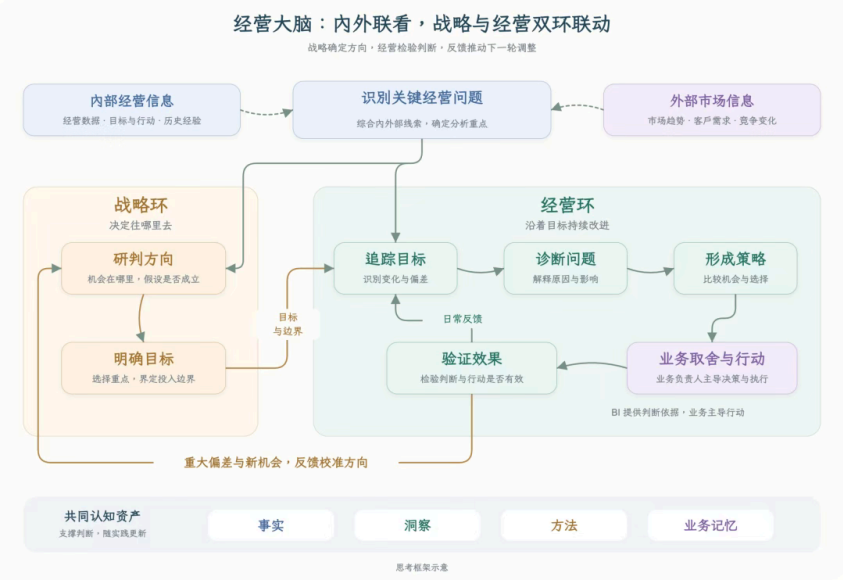


图19 高德本地生活经营分析大脑框架图

具体来说，我认为，作为BI，我们更应该关心这些问题：

- **眼前究竟该查什么？**收入增长放缓，先查供给与转化，还是客户预算和竞争变化？我想让系统**先根据业务背景确定问题——此刻该诊断问题还是提出新的策略方向——再选择分析能力**。

- **局部调整够不够？**经营环**跟进目标、诊断偏差、验证动作**；如果市场变化已经影响原来的假设，命题中枢会把问题路由回战略环，**重新审视方向和投入**。
- **这次选择能留下什么？**除了事实、洞察和方法，还应该留下业务记忆：为什么这样选，执行后发生了什么，下次遇到类似问题能参考哪些经验。

用一个假设场景说明：某项业务指标持续偏离目标，经营追踪先检查执行环节；如果进一步发现外部环境已经改变，原有增长假设不再成立，问题就该回到对于战略方向的重新审视——**此时需要调整的，可能是方向，而不只是动作。**

在这个过程中，**AI 可以扩大信息处理和分析的范围，BI 需要结合商业模式判断轻重，业务负责人作出取舍并推动行动**。

最近对于经营大脑框架的设计和持续思考，让我重新想了一遍：BI 应该站在经营的什么位置。

AI 时代的 BI，是经营判断能力的设计者，也是判断质量的守门人。

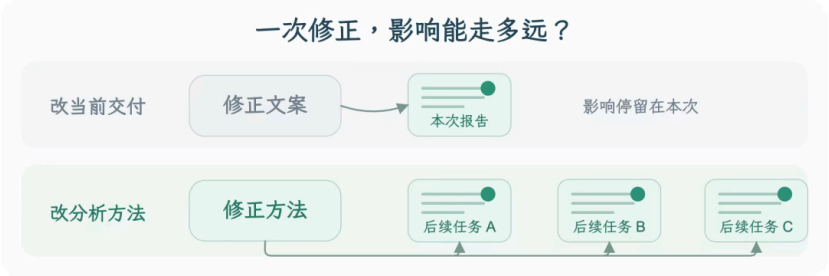
1.设计者的这一面，是**把对生意的理解沉淀成知识、方法和校验，让 AI 能稳定地产出可信的分析，让这一次的经验被下一次直接用；**
2.守门人的这一面，是**守住那些无法委托给 Agent 的环节——判断什么问题值得优先查，审阅系统的理解有没有偏离业务，在资源与策略的取舍上拿出专业意见。**

06
实践之后，我开始用不同的标准看待自己的工作

6.1 交付：从这一份报告，到下一次分析

经过 S1 的实践，我最直观的变化，是看报告的方式。最初审报告，我主要看归因够不够深、文字清不清楚、数字有没有错。后来遇到“分析太浅”，才发现有时缺证据，有时缺分析路径。把眼前这段文字改好，很可能只是暂时把问题盖住了。

现在**我会顺着反馈往前找，把该修的内容带回方法、知识或实现，再用案例复查**。下一次同类分析能不能因此做好，比这一份报告改了多少遍，更能说明能力有没有进步。我对 AI Native 交付的理解，也是在这里变得具体的。



6.2 经验：从自己会做，到能够讲清适用条件

要让下一次也能用，经验就得讲得比以前更清楚。写 Skill 和知识库时，我经常需要停下来确认：这个判断针对哪类对象，适用到哪个经营阶段，换个条件还成不成立？把“纳入统计”与“已经使用”混为一谈，就是省略这些前提可能造成的错误。

以前自己做分析，许多前提可以在脑子里补上。交给 AI 后，省略的部分就会变成误解。我得把依据、适用条件和反向信号写出来，也借这个过程检查自己有没有想当然的地方。**知识库写得越具体，我对业务理解里的缺口也看得越清楚**。



6.3 价值：从亲自控制细节，到判断哪里需要控制

但讲清楚经验，并不意味着每一步都要替模型规定好。前面那次规则越加越多、报告却没有变好的返工，也让我反省自己是不是管得太细了。对大模型幻觉的不放心，却成为了限制报告质量提升的脚铐。

现在我会把计算和引用交给程序核验，让模型尝试不同解释，再结合业务知识检查它的判断。比起逐句控制输出，我更需要判断哪些地方不能错、哪些地方可以探索，以及怎样知道探索有没有价值。

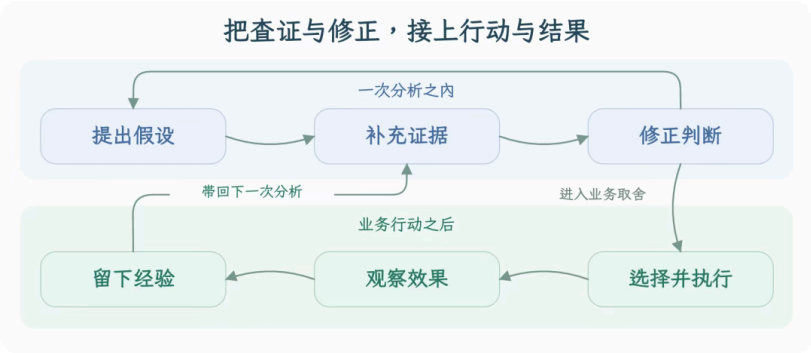
这也让我对 BI 这个岗位的转型有了实际的体会。理解业务仍然重要，但还要能把这种理解写成方法，放进产品和工程里，并通过真实分析检验它。我的工作范围变宽了，对专业判断的要求也更具体了。



6.4 下一步，把反馈接回来

带着这些认识，下一步我最想补的是反馈。现在已经能根据报告问题修改系统，但一次分析怎样自主追查、业务执行后的结果怎样回来，还没有完全接上：

- **Agentic loop：从一个经营问题出发，按发现补充证据、检查反证、调整路径，让分析能够继续深入。**
- **Human in the loop：让业务补充背景、参与取舍，再把动作和效果带回知识与后续分析。**



07
结语

写到最后，又想起第 1 节开头那句话：报告写完了，讨论才刚刚展开。这个 S1 给这句话添了一层新的身份：它是 BI 的日常处境，也是这套系统要抵达的地方——**问题被接着查清，动作被验证效果，经验长成下一次判断的底气**。那些报告之后还在继续的问题，就是我们作为 BI 要一直做下去的事。



欢迎留言一起参与讨论~

AI Native · 目录

< 上一篇 · AI Native 回忆录：稳定性前端 S1 实践复盘

阅读 3822

留言

写留言

还没有留言，来写首评